Figma MCP ハンズオン準備ドキュメント

> 月曜朝のハンズオンに向けた、Figma MCP × GitHub Copilot × Next.js の実装理解と手順整理

---

1. 状況整理

不安要素マッピング

```
構造理解の欠落
├── Figma MCP: 何ができて何ができないか
├── instructions: 効果の実感がない
├── ワークフロー: 開始から完了までの型がない
└── プロンプト/チャット運用: 粒度感覚がない
```

根本課題：個別の技術ではなく、「AIコーディングの型」そのものが未定着。Figma MCP の問題ではなく、コンテキストエンジニアリング全般の問題。そこを先に押さえるのが最短ルート。

---

2. 全体戦略

```
木夜   : 現状の棚卸し（何が分かっていないかを可視化）
金     : コンテキストエンジニアリング概念 + Figma MCP 仕様理解
土     : 一機能をゼロから再現（手順書の初稿作成）
日午前: 別機能で手順書を検証（他人が使えるか）
日午後: ハンズオン資料化 + 想定Q&A
月朝   : 本番
```

原則：書籍は土日には読まない。Figma MCP は新しすぎて体系的な書籍がなく、**一次情報（公式ドキュメント）> 実験 > 記事**の順で効率が高い。

---

3. 読むべき情報源

3.1 Figma MCP（日本語で読める一次情報）

優先	情報源	位置付け	所要
★★★	Figma MCPサーバーのガイド（Figma公式ヘルプ日本語版）	一次情報・公式	30分
★★★	Dev Mode MCPサーバーのご紹介（Figma公式ブログ日本語）	Figma の設計思想が分かる	20分
★★	Figma MCPの次の一手—3つのスキル（こぎそ氏 note）	2026年2月の最新機能（Skills）解説	15分
★★	Servithinkブログ：実装手順	日本語で最も網羅的な実装記事	20分

英語公式の developers.figma.com に日本語版はないため、ツールの詳細仕様（13個のツール一覧、パラメータ）だけは英語公式を参照。DeepL で十分読める。

3.2 コンテキストエンジニアリング（無料記事3本）

書籍（蒲生弘郷『コンテキストエンジニアリング』技術評論社、2026年2月刊、★4.3）は良書だが240ページあり、RAG/Agent 開発が中心でフロント実装には直接的でないため、ハンズオンには間に合わない。以下の記事で最小限の概念を獲得する。

優先	記事	得られるもの
★★★	コンテキストエンジニアリングの第一歩（窓の杜）	情報を入れすぎると精度が落ちる理由
★★★	コンテキストエンジニアリング入門（Speaker Deck）	「いい感じの中間表現」概念図
★★	日経xTECH記事	Anthropic の定義（有料会員のみ全文）

核心概念：

- コンテキスト = システム指示 + ツール + 外部データ + 履歴 + ユーザープロンプト
- **Lost in the Middle**：長いコンテキストの中央の情報は無視されやすい
- **Anthropic の定義**：「望ましい結果の最大化に必要な、最小限の高シグナルトークン」
- 引き算の技術。入れるより**削る**ほうが難しい

3.3 GitHub Copilot Instructions

優先	情報源	目的
★★	VS Code Custom Instructions公式	階層と Diagnostics の使い方
★	GitHub Docs: Repository custom instructions	設定方法の公式

---

4. Figma MCP の本質理解

4.1 位置付け

AI にデザインファイルの**構造化データ**（変数、コンポーネント、レイアウト、Auto Layout）を渡す仕組み。スクリーンショット推測ではなく、Figma の内部構造を直接読み込ませる点が従来との違い。

4.2 サーバーの種類

種類	接続先	特徴
Desktop (Local)	`http://127.0.0.1:3845/mcp`	Figma デスクトップアプリ必須、Dev/Full seat 必要
Remote	`https://mcp.figma.com/mcp`	Write to canvas 対応、対応クライアント限定

4.3 主要ツール（Copilot Chat で `#` で確認可能）

- `get_code`：選択フレームからコード生成
- `get_variable_defs`：デザイントークン（色、スペーシング、タイポ）抽出
- `get_design_context`：レイヤ構造取得
- `get_image`：スクリーンショット取得

4.4 出力品質を左右する3要素

```
1. Figma 側の整備
   ├── Variables（色/スペーシング/radius/typography）
   ├── Components（再利用される UI の明示）
   ├── Auto Layout（レスポンシブ意図）
   └── 意味のあるレイヤ名（Group 5 ではなく CardContainer）

2. Code Connect（オプション、最強）
   └── Figma コンポーネント ↔ コードコンポーネントの直接マッピング

3. instructions（IDE 側）
   └── プロジェクト規約・使用コンポーネント・命名規則
```

5. Copilot instructions の階層

レベル	ファイル	スコープ
Personal	GitHub Profile 設定	全体。個人設定が優先されやすい
Organization	GitHub UI	組織全体
Repository	`.github/copilot-instructions.md`	リポジトリ全体
Path-specific	`.github/instructions/*.instructions.md` + `applyTo`	特定ファイル/ディレクトリ
Prompt files	`.github/prompts/*.prompt.md`	手動呼び出し
AGENTS.md / CLAUDE.md	リポジトリ任意位置	Agent 用。常時適用

どれが効いているかの確認方法：VS Code で Chat ビュー右クリック → Diagnostics で**ロード中の instructions ファイル一覧が見られる**。これを金曜一番に確認すべき。

Next.js 特有の instructions に入れるべき項目

- App Router / Pages Router の明示
- Server Component と Client Component の使い分けルール（`'use client'` をいつ付けるか）
- スタイリング方針（Tailwind / CSS Modules / shadcn など）
- ディレクトリ構造（`app/` 配下の components 置き場）
- 画像は `next/image`、リンクは `next/link` 指定

6. 疑問への直接回答

6.1 instructions は Figma MCP 実装において重要か

結論：極めて重要。ただし Figma 側が整備されていない場合は最重要。

```
Figma側が整備されている（Variables/Components/Code Connect あり）
  → MCPから良質な構造化データが取れる
  → instructionsは「プロジェクト規約」（Next.js App Router, Tailwind等）を伝えるのが主目的

Figma側が半整備（今回のケースの可能性）
  → MCPから来るデータは部分的
  → instructionsで「不足分をどう埋めるか」をAIに指示する必要がある
  → instructionsの重要度が跳ね上がる
```

デザイナーが用意した Figma の整備度は、**土曜の最初に確認**する。確認方法：

1. Figma でデザイナー作成のフレームを選択
2. `#get_variable_defs` を叩く → 変数が返ってくるか
3. `#get_code` を叩く → 生成コードの中で既存コンポーネントを参照しているか

6.2 スタートから完了までの具体的な流れ

標準ワークフロー（Figma 整備済みの場合）：

```
① 準備フェーズ
   ├── .github/copilot-instructions.md 確認
   ├── Figmaデスクトップ起動 → Dev Mode MCP有効化
   └── VS Code Copilot Chat: Agent モード + Diagnosticsでinstructions確認

② コンテキスト取得フェーズ
   ├── Figmaで対象フレーム選択 or URL取得
   ├── Copilot Chat: 「#get_code このフレームのコンテキストを取得」
   └── 返ってきた構造（Variables/Components/Layout）を自分で確認

③ 実装計画フェーズ（チャット単位：1機能 = 1チャット）
   ├── 「このFigmaフレームを [該当パス] に Next.js App Router で実装したい」
   ├── 「既存の [類似コンポーネント名] と同じパターンで書いて」
   └── 分割が必要な場合、小さいコンポーネントから生成

④ 生成フェーズ
   ├── 生成コードをレビュー（Server/Client判定、Tailwind使い方）
   ├── Figmaデザインと差分確認
   └── 不足分は追加プロンプト

⑤ 統合フェーズ
   ├── 型定義追加、プロパティ整理
   ├── 既存ページに組み込み
   └── 動作確認
```

6.3 Figma MCP にできること・できないこと

	できる	できない
取得	Variables（色/間隔/文字）	画面遷移・ルーティング設計
取得	Components 階層・プロパティ	状態管理（Zustand/Redux 判断）
取得	Auto Layout 構造	API設計・データフェッチ戦略
取得	レイヤ名・ノードツリー	ビジネスロジック
取得	画像/アイコン書き出し	アニメーション詳細（hover/transition）
取得	Code Connect マッピング	アクセシビリティ判断
生成	React/Vue/HTML+CSS	ディレクトリ設計

要点：**見えているものの構造化データ**は取れるが、**見えない設計意図**は取れない。後者は instructions とプロンプトで補うのが仕事。

6.4 効果的なプロンプトの出し方

5要素テンプレート：

```
1. [対象] どのFigmaフレーム/リンク
2. [出力先] どのディレクトリ・ファイル名
3. [参照] 類似する既存コンポーネント名
4. [制約] Server/Client、型定義、テスト有無
5. [除外] やってほしくないこと（any禁止、inline style禁止など）
```

例：

> このFigmaフレーム（URL）を `app/products/[id]/components/ProductCard.tsx` に実装して。既存の `app/common/components/Card.tsx` と同じパターン（Server Componentベース、Tailwind、型はPropsで定義）で書いて。 画像は `next/image`、リンクは `next/link` を使うこと。 `any` 型と `useEffect` は使わないで。

よくない例：「このデザインを実装して」→ コンテキスト不足で AI が推測で埋めるため再現性ゼロ。

6.5 チャットの単位

結論：1機能 = 1チャット が基本。

状況	判断
新しいページ/機能の実装開始	新チャット（コンテキストクリーン）
同じコンポーネントの修正が続く	同チャット継続
5〜10往復して AI が混乱し始めた	**新チャット**（Lost in the Middle 回避）
別コンポーネントに移る	新チャット
Figma の別フレームを参照	新チャット推奨

チャットが長引いた時の兆候：同じ指摘を繰り返している、ファイルパスを間違え始める、既に修正した箇所を壊す。→ 即座に新チャット。

---

7. ハンズオン準備計画

7.1 今夜（木夜、60分）

#	タスク	目的	所要
1	現在の `.github/copilot-instructions.md` と MCP 設定ファイルを開いて読む	どの指示が効いているか把握	30分
2	VS Code Copilot Chat Diagnostics を開き、ロード中の instructions を確認	実際に効いているファイルの特定	10分
3	デザイナー作成の Figma で `#get_variable_defs` / `#get_code` を叩く	Figma 側整備度の把握	10分
4	土曜の再現対象機能を1つ選ぶ	ターゲット確定	10分

棚卸しの観点：

- Figma MCP が返すデータ構造（`get_code` vs `get_variable_defs` などの使い分け）
- Copilot instructions の階層（repo / personal / workspace）とどれが優先されるか
- 生成されたコードを自分がどこまで手で修正しているか
- Tailwind / CSS Modules / shadcn など、プロジェクトのスタイリング規約を instructions に書けているか
- Next.js の App Router / Pages Router どちらか、Server/Client Component の使い分け方針

7.2 金曜（2〜3時間）

```
AM ： コンテキストエンジニアリング無料記事3本（90分）
PM ： Figma公式ヘルプ日本語版 + mcp-server-guide（90分）
夜 ： 土曜の実験対象機能を1つ決定
```

7.3 土曜（フル）

```
午前 ： 対象機能のFigmaフレームで #get_variable_defs / #get_code 実行
        → 返ってくるデータの構造を自分で読み解く
午後 ： ゼロから手順書Ver.1作成。以下の粒度で書く
        ├── 事前準備（Figma側）
        ├── MCP起動確認（#get_design_context で疎通）
        ├── instructions の内容と意図
        ├── プロンプト例（具体的な文言）
        └── 生成後の手直しポイント
```

7.4 日曜（フル）

```
午前 ： 別機能で手順書を試す（「他人が追える」を確認）
        → 詰まった箇所が手順書の穴
午後 ： ハンズオン資料化
        ├── デモ用Figmaファイル準備
        ├── instructions サンプル（Next.js用）
        └── 想定Q&A
```

---

8. ハンズオンで刺さりそうな想定質問

1. 「instructions に書いたのに無視されるのはなぜ？」 → Personal が優先。Diagnostics で確認
2. 「生成コードが既存コンポーネントを使ってくれない」 → Code Connect か instructions で明示
3. 「毎回微妙に違う出力になる」 → Figma 側の Variables/命名の整備度で決まる
4. 「MCP ツールが IDE に出てこない」 → Figma デスクトップ再起動 → VS Code 再起動
5. 「どの単位で新しいチャットを始めるべき？」 → 1機能=1チャット。5〜10往復で混乱が見えたら新チャット
6. 「コンテキストを入れすぎると逆効果なのはなぜ？」 → Lost in the Middle。中央の情報は無視されやすい

---

9. 最重要アクション

今夜、`#get_variable_defs` を実際のデザイナー作成 Figma に叩いて何が返ってくるか確認する。

これで Figma 側整備度が分かり、instructions でどこまで補う必要があるかの判断がつく。手順書作成の前に、**自分が使っている道具の把握**がないと再現性のある手順書は書けない。